

Κατανεμημένα Συστήματα

1ο Φυλλάδιο εργαστηρίου 25/02/2026

Ροές Δεδομένων στην Java

Όπως και κάθε σύγχρονη γλώσσα προγραμματισμού, η Java προσφέρει ένα πλούσιο σύνολο κλάσεων και μεθόδων για διαχείριση αρχείων. Με τη Java μπορείτε να ανοίξετε αρχεία, να διαβάσετε και να γράψετε δεδομένα στα αρχεία και μάλιστα διατίθενται κλάσεις για να αυτοματοποιηθούν ορισμένες διαδικασίες, π.χ. αυτόματη αποθήκευση ενός πίνακα από String σε ένα αρχείο ή φιλτράρισμα των δεδομένων που διαβάζονται από ένα αρχείο.

Η είσοδος/έξοδος δεδομένων πραγματοποιείται μέσω αντικειμένων τα οποία ονομάζονται **ροές (streams)** τα οποία περιγράφουν ακριβώς ακολουθίες δεδομένων, bytes ή χαρακτήρων. Τονίζεται ότι μια ροή μπορεί να αναφέρεται στην ανάγνωση ή εγγραφή από ένα αρχείο, καθώς και από άλλους πόρους εισόδου/εξόδου όπως συνδέσεις δικτύου, εκτυπωτές, κ.λπ. Η Java παρέχει ένα πλήρες σύνολο κλάσεων για την προσπέλαση των δεδομένων ενός αρχείου και μάλιστα σε δύο επίπεδα, ένα σε επίπεδο byte (**byte streams**) και ένα σε επίπεδο χαρακτήρων (**character streams**). Όπως μαρτυρά και το όνομα τους, τα byte streams χρησιμοποιούνται για την επεξεργασία των δεδομένων των αρχείων, για ανάγνωση και εγγραφή και ειδικά όταν τα δεδομένα δε μπορούν να απεικονιστούν ως κείμενο (π.χ. εικόνα, ήχος). Για την περίπτωση αρχείων κειμένου, τα character streams προσφέρουν επιπλέον λειτουργίες, όπως απευθείας κωδικοποίηση σε κάποιο σύνολο χαρακτήρων (π.χ. ISO-8859-7, UTF-8). Σε μερικές περιπτώσεις μάλιστα και ειδικά για αρχεία κειμένου, τα character streams αποδίδουν καλύτερα από τα byte streams.

Για να δημιουργήσουμε μια ροή δεδομένων, αρχικά θα χρησιμοποιήσουμε την κατάλληλη κλάση που εξαρτάται από την συσκευή που βρίσκεται στην άλλη μεριά της ροής μας και τον τύπο των δεδομένων που θέλουμε να χειριστούμε. Παραδείγματος χάριν, για την περίπτωση των δυαδικών αρχείων οι κατάλληλες κλάσεις είναι οι *FileInputStream* και *FileOutputStream*.

Η κύρια λειτουργία των βασικών ροών εισόδου/εξόδου είναι να μεταφέρουν τα δεδομένα όπως ακριβώς δίνονται στις μεθόδους τους ένα-ένα byte κάθε φορά χωρίς κάποια επεξεργασία. Για να αυξήσουμε λοιπόν την λειτουργικότητα μίας βασικής ροής "τυλίγουμε" (wrap) γύρω της μια ανώτερου επιπέδου ροή. Αυτή η κατηγορία των κλάσεων αυξάνει την αποδοτικότητα του προγράμματός μας καθώς μπορεί να κάνει buffering των δεδομένων ή να τα επεξεργαστεί κατάλληλα. Παράδειγμα μιας τέτοιας ροής είναι οι κλάσεις *BufferedInputStream* και *BufferedOutputStream*, που χρησιμοποιούν ενδιάμεση μνήμη (buffer) για να αυξήσουν την απόδοση της εισόδου/εξόδου δεδομένων. Το "τύλιγμα" της βασικής ροής επιτυγχάνεται με το να την περάσουμε σαν παράμετρο στον κατασκευαστή (constructor) της ανώτερης ροής.

Ακολουθούν παραδείγματα για ανάγνωση και εγγραφή χαρακτήρων σε αρχείο.

Ροές χαρακτήρων (Character Streams)

Η σύγχρονη Java χρησιμοποιεί τη δομή try-with-resources, η οποία κλείνει αυτόματα τα streams, ακόμη και αν συμβεί κάποιο σφάλμα.

1ο Παράδειγμα - Ανάγνωση και Εγγραφή

```
import java.io.*;
```

```
public class TextDemo {
    public static void main(String[] args) {

// Ανάγνωση ανά γραμμή
try (BufferedReader in = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = in.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Σφάλμα ανάγνωσης: " + e.getMessage());
}

// Εγγραφή από το πληκτρολόγιο
try (BufferedWriter out = new BufferedWriter(new FileWriter("output.txt"));
    BufferedReader reader = new BufferedReader(new InputStreamReader(System.in))) {
    System.out.print("Δώστε κείμενο: ");
    String text = reader.readLine();
    out.write(text);
} catch (IOException e) {
    System.err.println("Σφάλμα εγγραφής.");
}
    }
}
```

Επεξεργασία με Java 8+ Streams API

2ο Παράδειγμα

Η μέθοδος *readAllLines* (*static List<String> readAllLines(Path p, Charset set*) της κλάσης *Files* επιστρέφει μία λίστα από *String* αντικείμενα, το κάθε ένα από τα οποία θα περιέχει μια γραμμή του κειμένου.

Η μέθοδος *write* (*static Path write(Path p, Iterable<String> lines, Charset set, OpenOption ... options*) επιτρέπει να γράψουμε στο αρχείο που καθορίζεται στην 1^η παράμετρο, το περιεχόμενο ενός *iterable* αντικειμένου τύπου συμβολοσειράς.

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;
import java.io.IOException;

final class Main {
    public static void main(String args[]) {
        try {
            List<String> lines = Files.readAllLines(Paths.get("Letter.txt"));
            lines.add("Προσθήκη νέας γραμμής");
            Files.write(Paths.get("NewLetter.txt"), lines);
        } catch (IOException ex) {ex.toString(); }
    }
}
```

Εναλλακτικά μπορούμε να έχουμε το ίδιο αποτέλεσμα χρησιμοποιώντας τη μέθοδο *lines* η οποία επιστρέφει ένα *stream* από τις γραμμές που εμπεριέχονται στο αρχείο.

```
import java.nio.file.Files;
```

```
import java.nio.file.Paths;
import java.util.List;
import java.io.IOException;
import java.util.stream.Stream;

final class Main {
    public static void main(String args[]) {
        try (Stream<String> stream = Files.lines(Paths.get("Letter.txt"))) {
            stream.forEach(System.out::println);

            // Εναλλακτικά: stream.forEach(n->System.out.println(n));
        } catch (IOException e) {
            e.toString();
        }
    }
}
```

1η Εργαστηριακή Άσκηση

Περιγράψτε τον ακόλουθο κώδικα. Ποιο θα είναι το αποτέλεσμα της εκτέλεσης του προγράμματος, αν το αρχείο κειμένου «Letter.txt» περιέχει το παρακάτω κείμενο:

*Me gusta la moto, me gustas tú
Me gusta correr, me gustas tú!
Me gusta la lluvia, me gustas tú!
Me gusta volver, me gustas tú.
Qué voy a hacer? Je ne sais pas.
Qué voy a hacer? Je ne sais plus.*

```
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;

public class StreamLab {
    public static void main(String[] args) {
        try (Stream<String> lines = Files.lines(Paths.get("Letter.txt"))) {
            List<String> result = lines
                .filter(line -> line.startsWith("Me gusta"))
                .filter(line -> !line.endsWith("!"))
                .map(String::toUpperCase)
                .sorted()
                .map(line -> line.substring(0, 16))
                .collect(Collectors.toList());

            result.forEach(System.out::println);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Απάντηση

Το πρόγραμμα θα εμφανίσει :

ME GUSTA LA MOTO
ME GUSTA VOLVER,

Με την μέθοδο **filter()** φιλτράρουμε τα δεδομένα με βάση συγκεκριμένους κανόνες που ορίζουμε ως παράμετρο. Η **map()** μέθοδος της Stream κλάσης χρησιμοποιείται για τον μετασχηματισμό ενός αντικειμένου σε άλλη μορφή με την εφαρμογή μιας συνάρτησης (στο παράδειγμα μας της συνάρτησης `toUpperCase` και `substring`).

Με την **sorted()** ταξινομούμε τα δεδομένα του stream που προκύπτουν από την προηγούμενη επεξεργασία και με την **toList()** μεταφέρουμε τα τελικά δεδομένα του stream σε μια λίστα (στο παράδειγμα μας στην λίστα με όνομα `list`).

Ροές από bytes

Οι κλάσεις οι οποίες αναπαριστούν ροές από bytes είναι υποκλάσεις των κλάσεων **InputStream** και **OutputStream**, αντίστοιχα. Υποκλάσεις των παραπάνω οι οποίες αναφέρονται σε ροές αρχείων είναι οι **FileInputStream** και **FileOutputStream**, αντίστοιχα. Το άνοιγμα μιας τέτοιας ροής γίνεται με χρήση του αντίστοιχου κατασκευαστή. Έτσι, το άνοιγμα ενός αρχείου για ανάγνωση μέσω μιας ροής από bytes γίνεται με την εντολή:

```
FileInputStream input = new FileInputStream("data.dat");
```

όπου `data.dat` είναι το όνομα του αρχείου το οποίο θέλουμε να ανοίξουμε για ανάγνωση. Το άνοιγμα ενός αρχείου για εγγραφή μέσω μιας ροής από bytes γίνεται με την εντολή:

```
FileOutputStream output = new FileOutputStream("data.dat");
```

όπου `data.dat` είναι το όνομα του αρχείου το οποίο θέλουμε να ανοίξουμε για εγγραφή. Η ανάγνωση ενός byte από το αρχείο γίνεται με χρήση της μεθόδου ***byte read()*** της κλάσης `InputStream` (άρα και της `FileInputStream`) η οποία επιστρέφει το *επόμενο* byte στη ροή, ξεκινώντας από το πρώτο (το επιστρέφει με τη μορφή `int`). Όταν η ανάγνωση φτάσει στο τέλος της ροής (αρχείο στην προκειμένη περίπτωση) η `read()` επιστρέφει την τιμή `-1`.

Η *εγγραφή* ενός byte σε ένα αρχείο γίνεται με χρήση της μεθόδου ***write(byte b)*** της κλάσης `OutputStream` (άρα και της `FileOutputStream`) η οποία εγγράφει το byte `b` του ορίσματος της στη ροή.

Μετά την ολοκλήρωση της προσπέλασης σε μια ροή είναι απαραίτητη η αποδέσμευση του σχετικού πόρου. Αυτό γίνεται με τη μέθοδο `close()` των κλάσεων `InputStream` και `OutputStream`.

Όλες οι παραπάνω μέθοδοι και κατασκευαστές εγείρουν, σε περίπτωση σφάλματος, μια εξαίρεση του τύπου `IOException`.

3ο Παράδειγμα

Υλοποιήστε πρόγραμμα που αντιγράφει τα περιεχόμενα ενός αρχείου (`File1.txt`) σε ένα άλλο αρχείο (`File2.txt`). Η ανάγνωση και η εγγραφή θα γίνει ανά byte.

```
import java.io.*;  
  
public class FileCopy {  
    public static void main(String[] args) {  
        try (FileInputStream fin = new FileInputStream("source.dat");  
            FileOutputStream fout = new FileOutputStream("dest.dat")) {  
  
            int i;  
            while ((i = fin.read()) != -1) {
```

```

        fout.write(i);
    }
} catch (IOException e) {
    System.out.println("I/O Error: " + e.getMessage());
}
}
}

```

Εγγραφή/Ανάγνωση αντικειμένου σε μια ροή (Serialization)

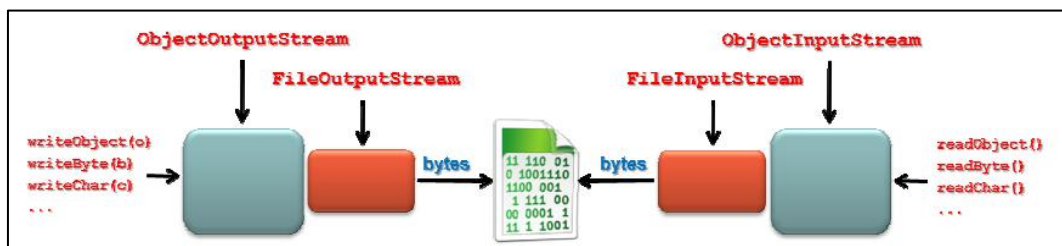
Επιπλέον των χαρακτήρων ή άλλων πρωταρχικών (int, double, float, byte, ...) δεδομένων που μπορούμε να γράψουμε ή να διαβάσουμε σε/από αρχεία ή οποιοδήποτε άλλου τύπου stream, η Java επιτρέπει την εγγραφή και ανάγνωση αντικειμένων. Για να το επιτύχουμε αυτό θα πρέπει να χρησιμοποιήσουμε την κλάση *ObjectOutputStream* για να εγγράψουμε ένα αντικείμενο της Java σε ένα *OutputStream*. Ομοίως θα χρησιμοποιήσουμε την κλάση *ObjectInputStream* για να διαβάσουμε ένα αντικείμενο από κάποιο *InputStream*, που έχει γραφεί προηγουμένως με την *ObjectOutputStream*.

Η κλάση *ObjectOutputStream* παρέχει μεθόδους για την εγγραφή τόσο βασικών τύπων όσο και αντικειμένων με τη μορφή δυαδικής αναπαράστασης σε ένα ρεύμα εξόδου. Διαθέτει για κάθε βασικό τύπο μία μέθοδο της μορφής *writeX(X)*, δηλαδή *writeInt(int)*, *writeBoolean(boolean)*. Στη κλάση αυτή ορίζεται και η μέθοδος ***void writeObject (Object obj)***, η οποία παίρνει ως παράμετρο το αντικείμενο που θέλουμε να γράψουμε στο stream της εξόδου.

Αντίστοιχα η κλάση *ObjectInputStream* παρέχει μεθόδους για την ανάγνωση τόσο βασικών τύπων όσο και αντικειμένων με τη μορφή δυαδικής αναπαράστασης από ένα ρεύμα εισόδου. Οι μέθοδοι αυτές έχουν για τους βασικούς τύπους τη μορφή *readX(X)*, δηλαδή *readInt()*, *readByte()*. Επίσης ορίζεται η μέθοδος ***Object readObject()***, η οποία επιστρέφει το αντικείμενο που είχαμε γράψει προηγουμένως στο stream. Όπως βλέπουμε η μέθοδος **επιστρέφει αντικείμενο τύπου Object** και επομένως το αντικείμενο αυτό θα πρέπει να το κάνουμε cast στον πραγματικό τύπο του αντικειμένου.

Οποιοδήποτε αντικείμενο μπορεί να περαστεί ως όρισμα στην μέθοδο *writeObject()* αρκεί η κλάση να υλοποιεί το interface *java.io.Serializable*. Το συγκεκριμένο interface δεν περιέχει κάποια μέθοδο, αλλά λειτουργεί ως ένδειξη (*marker*) ότι τα αντικείμενα κλάσεων που το υλοποιούν μπορούν να εγγραφούν/αναγνωστούν σε streams. Επίσης μπορούν να χρησιμοποιηθούν πίνακες ή String, που υλοποιούν το συγκεκριμένο interface. Μια *Serializable* κλάση πρέπει να περιέχει μόνο πρωταρχικούς τύπους δεδομένων ή αναφορές σε αντικείμενα κλάσεων που υλοποιούν το συγκεκριμένο interface. Τα μόνα πεδία που εξαιρούνται από το serialization είναι τα static και τα transient.

Στο ακόλουθο σχήμα απεικονίζεται η διαδικασία εγγραφής και ανάγνωσης αντικειμένων αλλά και πρωταρχικών τύπων προς/από ένα δυαδικό αρχείο. Είναι φανερό ποιες είναι οι κλάσεις που θα πρέπει να συνεργαστούν για αυτή τη λειτουργία:



Σε ένα stream μπορούμε να εγγράψουμε πολλαπλά αντικείμενα, κατά την ανάγνωση θα πρέπει να τα διαβάσουμε με τη σειρά που τα γράψαμε και να τα κάνουμε κατάλληλα casting.

4ο Παράδειγμα

```
import java.io.*;
public class MainOut { // εγγραφή αντικειμένων σε αρχείο

    public static void main(String[] args) {
        ObjectOutputStream out;
        int array[] = {1,4,5,8};
        try{
            out=new ObjectOutputStream(new FileOutputStream("object.dat"));

            out.writeObject(array);
            out.writeObject("abc");
            out.writeObject(new PhoneBook("Nikos", "2273012345"));
            out.writeObject(new PhoneBook("Mixalis", "2273054321"));
            out.flush();
            out.close();

            System.out.println("Object written to file");
        }
        catch (FileNotFoundException ex) {
            System.out.println("Error with specified file") ;
            ex.printStackTrace();
        }
        catch (IOException ex) {
            System.out.println("Error with I/O processes") ;
            ex.printStackTrace();
        }
    }
}

import java.io.Serializable;

public class PhoneBook implements Serializable{ //serializable κλάση
    private String name;
    private String phone;

    public PhoneBook(String n, String p){
        this.name = n;
        this.phone = p;
    }

    public String toString (){
        return "the subscriber "+ this.name+ " has the number :"+this.phone;
    }
}
```

2^η εργαστηριακή άσκηση – Αρχεία αντικειμένων

Υλοποιήστε κλάση που θα διαβάζει τα αντικείμενα από το αρχείο που δημιουργήσαμε στο προηγούμενο παράδειγμα και θα εμφανίζει τα περιεχόμενα του στην οθόνη.

Απάντηση

```
import java.io.*;
public class MainIn { // ανάγνωση αντικειμένων από αρχείο

    public static void main(String[] args) {
        ObjectInputStream in = null;
        int array1[] = null;
        String str = null;
        PhoneBook book;

        try{
            in = new ObjectInputStream(new FileInputStream("object.dat"));

            array1 = (int []) in.readObject();
            str = (String) in.readObject();

            while((book = (PhoneBook) in.readObject()) != null){
                System.out.println(book);
            }

        } catch (EOFException ex) {
            System.out.println("End of file reached.");
        } catch (ClassNotFoundException ex) {
            System.out.println("Error casting") ;
            ex.printStackTrace();
        } catch (FileNotFoundException ex) {
            System.out.println("Error specified file does not exist") ;
            ex.printStackTrace();
        } catch (IOException ex) {
            System.out.println("Error with I/O processes") ;
            ex.printStackTrace();
        }

        finally {
            try{
                in.close();
            } catch(IOException ex){
                System.out.println("Another IOException during the closing");
                ex.printStackTrace();
            }
        }

        System.out.println("Array contains :");
        for(int i=0; i<array1.length; i++){
            System.out.print(array1[i]+" ");
        }

        System.out.println("\nString contains : " +str);
    }
}
```

3^η εργαστηριακή άσκηση – Αρχεία αντικειμένων

Ο παρακάτω κώδικας προσπαθεί να γράψει στο αρχείο "CharacterObjects.dat" δυο αντικείμενα της κλάσης Character, είναι όμως ελλιπής και λανθασμένος. Ξαναγράψτε τον κώδικα ώστε να λειτουργεί σωστά προσθέτοντας ή τροποποιώντας εντολές εκτός της εντολής 1. Η τρέχουσα τιμή του πεδίου power ΔΕΝ θα πρέπει να αποθηκεύεται στην ροή.

```
import java.io.*;
public class Character {
    private String type;
    private int power;
    public Character(String t, int p) {type=t; power=p;}

    public static void main(String args[]) {
        Character ch1 = new Character(); // Εντολή 1
        Character ch2 = new Character("Fictional", 20);
        ObjectOutputStream os = new ObjectOutputStream("CharacterObjects.dat");
        os.writeObject(ch1, ch1);
        os.close();
    }
}
```

Απάντηση

```
import java.io.*;
public class Character implements Serializable {
    private String type;
    transient private int power;

    public Character() {}
    public Character(String t, int p) {type=t; power=p;}

    public static void main(String args[]) {
        Character ch1 = new Character(); // Εντολή 1
        Character ch2 = new Character("Fictional", 20);
        ObjectOutputStream os = null;

        try {
            os = new ObjectOutputStream(new FileOutputStream("CharacterObjects.dat"));
            os.writeObject(ch1);
            os.writeObject(ch2);
        }
        catch (IOException ioex) {System.out.println("IO Error!!!");}
        finally {try {if (os!=null) os.close();} catch (IOException ioex) {}}
    }
}
```